INSTITUTO TECNOLÓGICO SUPERIOR DE TEPOSCOLULA

INGENIERIA EN SISTEMAS COMPUTACIONALES

INVESTIGACION

DOCENTE: ING. MARCO ANTONIO RUIZZ VICENTE

UNIDAD 2

SISTEMA OPERATIVOS

ALUMNO: ABEL LOPEZ RAMIREZ

MATRICULA: 2420225

SEMESTRE:V GRUPO: S5A

SAN PEDRO Y SAN PABLO TEPOSCOLULA, OAXACA.
01 DE OCTUBRE DE 2026.

Hilos en Java, concurrencia y sincronización

1. Qué es un proceso ligero, hilo o thread

Un hilo es la unidad más pequeña de ejecución dentro de un programa. Se le llama proceso ligero porque comparte el mismo espacio de memoria y los mismos recursos con los demás hilos del mismo proceso, a diferencia de los procesos pesados del sistema operativo, que cuentan con memoria independiente.

La ventaja principal es que permite la concurrencia y el paralelismo, ya que se pueden ejecutar varias tareas al mismo tiempo, como atender varias casetas de peaje de forma simultánea. Además, crear un hilo y cambiar de un hilo a otro consume menos recursos que hacerlo con procesos completos.

2. Diferencia entre extender de Thread e implementar Runnable

Extender la clase Thread obliga a heredar directamente de ella. Su desventaja es que Java no admite herencia múltiple, por lo que la clase ya no puede heredar de ninguna otra.

Implementar la interfaz Runnable solo exige definir el método run. Su ventaja es que la clase puede heredar de otra clase si es necesario y que separa la lógica de la tarea de la ejecución del hilo. Es la práctica recomendada en Java.

3. Diferencia entre start y run

- El método start crea un nuevo hilo de ejecución independiente en el sistema operativo y llama internamente al método run dentro de ese nuevo hilo, lo que produce una ejecución concurrente.
- El método run ejecuta su contenido directamente en el hilo actual, que normalmente es main, de forma secuencial y sin crear ningún hilo nuevo.

Por esta razón, para que una tarea realmente se ejecute en paralelo siempre se debe invocar start y nunca run. Además, un mismo hilo solo puede iniciarse una vez, de lo contrario Java lanza una excepción.

4. Para qué sirve el método join

El método join hace que el hilo actual, generalmente el hilo principal main, espere a que termine la ejecución del hilo sobre el cual se invoca.

En el ejercicio de las casetas, llamar a join sobre cada hilo asegura que el reporte final no se imprima hasta que todas las casetas hayan procesado sus vehículos. Sin join, el hilo principal podría mostrar un total incompleto porque continuaría su ejecución sin esperar a los demás.

5. Qué es una condición de carrera o race condition

Ocurre cuando dos o más hilos intentan acceder y modificar un recurso compartido, como la variable totalRecaudadoGlobal, al mismo tiempo y sin sincronización.

En el código del reto, al no usar synchronized ni AtomicInteger, las operaciones de lectura y escritura de la instrucción totalRecaudadoGlobal += 50 se solapan entre los hilos. Esto provoca que el valor final sea inconsistente o menor al esperado, por la pérdida de datos debida a escrituras concurrentes.

Esto sucede porque la suma no es una operación atómica. En realidad consta de tres pasos, leer el valor actual, sumarle 50 y escribir el resultado. Si dos hilos leen el mismo valor antes de que cualquiera escriba, uno de los dos incrementos se pierde.

6. Mecanismos para evitar la condición de carrera en Java

- Bloque synchronized, que garantiza que solo un hilo a la vez pueda ingresar a la sección crítica de código.
- Palabra reservada volatile, que garantiza la visibilidad de los cambios en las variables entre diferentes hilos, evitando que cada hilo use una copia en caché local. Por sí sola no vuelve atómicas operaciones como la suma.
- Clases atómicas del paquete java.util.concurrent.atomic, como AtomicInteger, que ofrecen operaciones atómicas seguras para hilos sin necesidad de bloqueos explícitos, por ejemplo el método incrementAndGet.

7. Ciclo de vida de un hilo en Java

- Nuevo o New. El hilo ha sido creado con new Thread pero todavía no se ha llamado al método start.
- Ejecutable o Runnable. El hilo está listo para ejecutarse o se está ejecutando.
- Bloqueado, en espera o en espera temporal, conocidos como Blocked, Waiting y Timed Waiting. El hilo está suspendido de forma temporal, por ejemplo al ejecutar Thread.sleep o al esperar la liberación de un bloqueo.
- Terminado o Terminated. El hilo completó su método run o finalizó a causa de una excepción.

Conclusión

Los hilos permiten aprovechar mejor los recursos y atender varias tareas a la vez, pero exigen cuidado al compartir datos. Usar Runnable, iniciar los hilos con start, esperar su finalización con join y proteger los recursos compartidos con synchronized o con clases atómicas son las bases para escribir programas concurrentes correctos y predecibles.